

PENETRATION TESTING REPORT

Mediroza General Hospital — Web Application Security Assessment

Prepared By:	Adzido Godsway (Junior Penetration Tester)
Organisation:	Networkwalks
Target URL:	https://medirozahospital.com
Assessment Type:	Full Black-Box Penetration Test
Timeline:	5 Days (Independent Execution)
Classification:	Confidential

1. Executive Summary

I was assigned to conduct a black-box penetration test on the web infrastructure of Mediroza General Hospital at <https://medirozahospital.com>. The client provided written authorisation for this assessment. The goal was to identify vulnerabilities, demonstrate their real-world impact through controlled exploitation, and provide actionable recommendations to improve the overall security posture of the organisation.

During the assessment, I identified seven vulnerabilities ranging from Medium to Critical severity. The most significant finding was an authentication bypass vulnerability caused by SQL injection on the web portal login endpoint, which allowed me to bypass authentication entirely without knowing valid user credentials. This granted access to three confidential patient lab report PDFs. After cracking the PDF encryption using NetworkWalks password cracking tools, I discovered sensitive metadata inside one of the files that pointed directly to a forgotten database backup stored in a publicly accessible folder on the server (/old).

This unencrypted database backup contained full employee records, monthly salaries of all hospital staff, bank account numbers, and ownership details of all company shareholders. Exposing this information violates data protection standards (including PCI-DSS and healthcare privacy guidelines) and leaves staff vulnerable to targeted phishing, identity theft, and extortion.

Overall Risk Rating: CRITICAL

Immediate remediation is strongly recommended before real patient data is stored or processed on the platform.

2. Scope and Methodology

2.1 Scope

The assessment was strictly limited to the target web infrastructure agreed with the client:

- **Target Domain:** <https://medirozahospital.com>

Explicit Exclusions & Rules of Engagement: Social engineering, Denial of Service (DoS/DDoS) attacks, and any security testing outside the designated domain were strictly prohibited.

Complete confidentiality was maintained throughout the test.

2.2 Methodology

A structured 4-phase black-box penetration testing methodology was followed:

- **Reconnaissance:** Passive and active information gathering using network tools, server fingerprinting, and standard web requests.
- **Vulnerability Identification:** Analyzing application endpoints especially authentication, parameter responses, and input validation to discover security weaknesses.
- **Exploitation:** Demonstrating real-world operational impact by exploiting vulnerabilities in a controlled, non-destructive manner.
- **Documentation:** Recording findings, compiling evidence/proof-of-concept steps, and outlining mitigation recommendations.

2.3 Tools Used

- **Burp Suite:** Web proxy for traffic interception, HTTP packet behavior analysis, and parameter testing.
- **WhatWeb & Whois:** Reconnaissance tools for technology stack and domain discovery.
- **NetworkWalks Hash Calculator:** Custom tool used to calculate and extract password hashes from encrypted PDF files.
- **Networkwalks Password Cracker / John the Ripper (JTR) wordlists:** Tools for cracking PDF file passwords via dictionary attacks.
- **ExifTool:** Command-line utility for reading and extracting hidden metadata from documents.
- **qpdf:** Command-line tool used to decrypt password-protected PDF files after cracking.
- **wget / cURL:** Command-line utilities for sending HTTP requests and retrieving remote server files.
- **Kali Linux Operating System:** Primary penetration testing execution platform.

3. Findings and Proof of Exploitation

3.1 Vulnerability Summary Table

#	Vulnerability Description	Location / Endpoint	Risk Level
1	Username enumeration via login error messages	patient/login.php & staff/login.php	Medium
2	SQL injection authentication bypass	patient/login.php	Critical
3	Confidential medical PDFs accessible post-bypass	patient/reports/	High
4	Weak PDF encryption passwords crackable via wordlist	patient_report_1.pdf	High
5	Sensitive metadata left inside patient PDF files	patient_report_3.pdf	Medium
6	Forgotten backup folder with directory listing enabled	/old/	Critical
7	Confidential staff salaries, bank details & shareholder data in plain text	/old/mediroza_db_backup_2019.sql	Critical

3.2 Finding 1 — Username Enumeration on Login Endpoints

Risk Rating: Medium | **Location:** patient/login.php

Description: Username enumeration occurs when an authentication endpoint reveals whether a username exists within the system database by returning distinct error responses for an invalid username versus an incorrect password. Secure authentication endpoints must return a consistent, generic error message for all failed authentication attempts to prevent account harvest attacks.

Steps Taken:

1. Identified the patient login forms during initial endpoint reconnaissance after reviewing disallowed paths in /robots.txt.
2. Intercepted login requests using Burp Suite and transmitted crafted payload packets to observe server response behavior.
3. Submitted an arbitrary non-existent username: username: hello | password: test123 -> Response: 'username not found'.

4. Submitted a standard administrative username: username: admin | password: -> Response: 'password incorrect'.

Impact: The application confirmed that the 'admin' account exists in the database, reducing attacker effort to password cracking or injection.

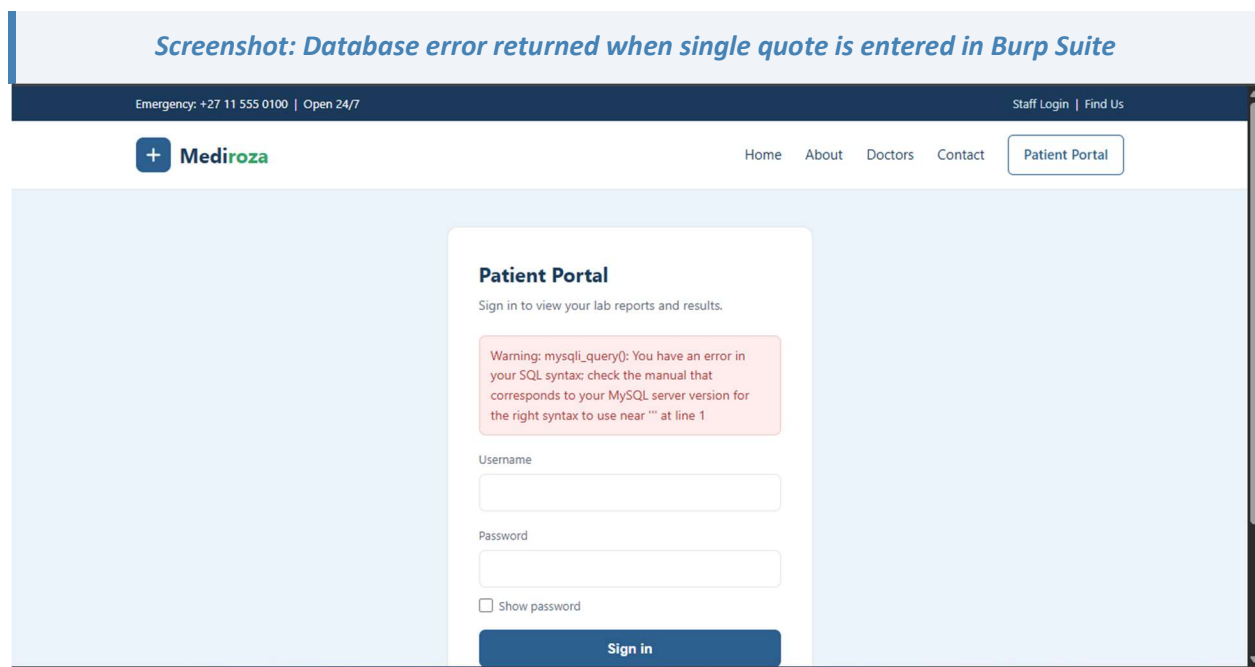
3.3 Finding 2 — SQL Injection Authentication Bypass

Risk Rating: Critical | **Location:** patient/login.php

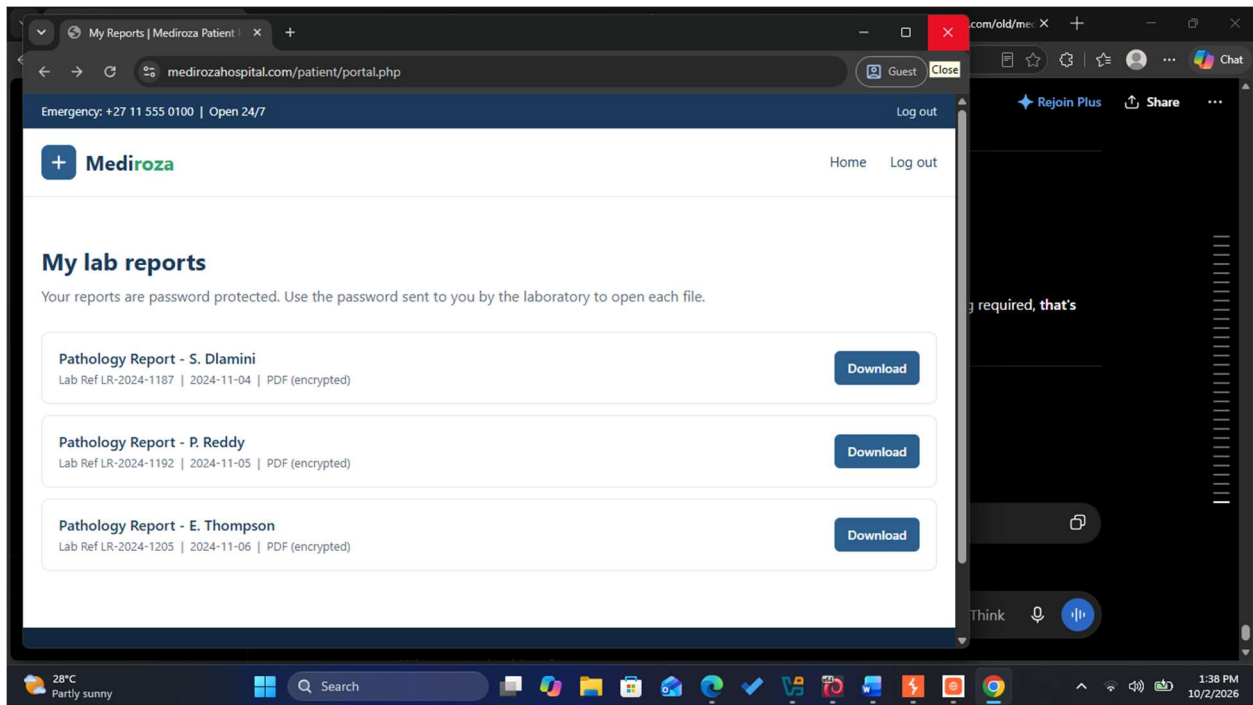
Description: SQL injection occurs when user-supplied input is directly concatenated into dynamic SQL queries without sanitization or parameterization. An attacker can input SQL control syntax to manipulate query execution logic. Here, improper constraints on the authentication form allowed commenting out the password validation entirely.

Steps Taken:

1. Tested for SQL injection by appending a single quote (') to the username field: username: admin' | password: .
2. The LiteSpeed web server returned a MySQL error: Warning: mysqli_query(): You have an error in your SQL syntax...
3. Crafted the authentication bypass payload utilizing SQL comment characters (--): username: admin' -- | password: anything.
4. The backend query structure was altered to comment out password checks, returning the admin session and granting full administrative portal access.



[Screenshot: Logged-in portal dashboard after executing SQL injection bypass]



3.4 Finding 3 — Confidential Medical PDFs Accessible After Login Bypass

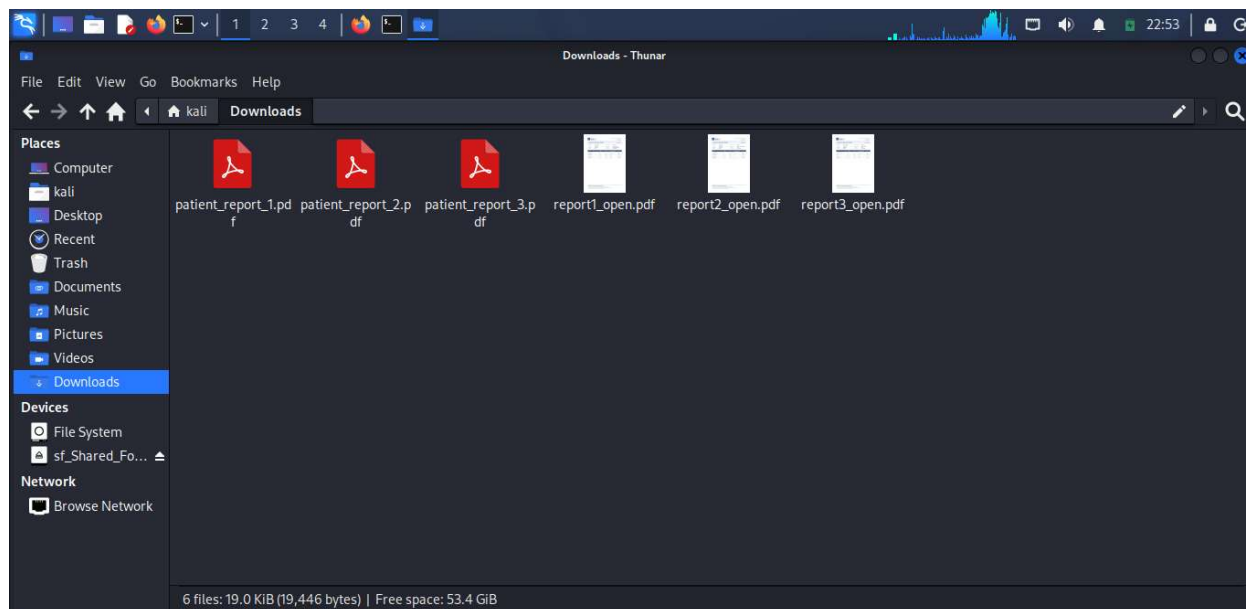
Risk Rating: High | **Location:** patient/reports/

Description: Gaining unauthorized portal access revealed three downloadable patient lab report PDF files (patient_report_1.pdf, patient_report_2.pdf, and patient_report_3.pdf). These documents contain confidential patient healthcare data that must be protected against unauthorized access.

Steps Taken:

1. Navigated the authenticated patient portal menu following SQL injection bypass.
2. Located three downloadable PDF patient report files in the reports directory.
3. Downloaded all three PDF files locally to the testing workstation for cryptographic inspection.

[Screenshot: Downloaded patient report files stored in testing directory]



3.5 Finding 4 — Weak PDF Passwords Crackable via Wordlist

Risk Rating: High | **Location:** patient_report_*.pdf

Description: Although the downloaded PDF files were password protected, the encryption keys used were extremely weak and easily guessable strings. Applying default wordlists enabled rapid offline decryption, rendering document protection ineffective.

Steps Taken:

1. Processed PDF files through the Networkwalks Hash Calculator to extract password hashes.
2. Executed dictionary attacks using Networkwalks Password Cracker.
3. Reports 1 and 2 cracked instantly using the built-in 100-word list: Report 1 Password: '123456' | Report 2 Password: 'password'.
4. Report 3 required a secondary run using the John the Ripper (JTR) default dictionary list: Report 3 Password: '!@#\$\$%^&' (Shift + 1234567).
5. Decrypted all files using qpdf and verified patient medical report contents.

[Screenshot: Networkwalks Password Cracker output displaying cracked passwords for Reports 1 and 2]

My Reports | M | X patient_report_ X patient_report_ X patient_report_ X Hash Calculator: X Password Crac: X Password Crac: X Password Crac: X + -

https://networkwalks.com/password-cracker/

Tried: 12 / 3556 1 pm/s 0%

```
[~] Trying: #!comment: X
[~] Trying: #!comment: This list is based on passwords most commonly seen on a set of Unix X
[~] Trying: #!comment: systems in mid-1990's, sorted for decreasing number of occurrences X
[~] Trying: #!comment: (that is, more common passwords are listed first). It has been X
[~] Trying: #!comment: revised to also include common website passwords from public lists X
[~] Trying: #!comment: of "top N passwords" from major community website compromises that X
[~] Trying: #!comment: occurred from 2006 onwards. X
[~] Trying: #!comment: X
[~] Trying: #!comment: Last update: 2025 (3546 entries) X
[+] MATCH 123456 ✓
```

PASSWORD CRACKED SUCCESSFULLY

123456

This is the PDF password. Open your PDF and enter it to unlock the file.

Copy password

PDF HASH (\$PDF\$...)

```
$pdf$2*3*128*4294967292*1*32*3361663365326235643333353531613238303137316238333238373763353339*32*ef16c52ab
8efce2c18c79e9d28895b5928bf4e5e4e758a4164004e56ffa0108*32*c431fab9cc5ef7b59c244b61b745f71ac5ba427b1b9102d
a0108*32*c431fab9cc5ef7b59c244b61b745f71ac5ba427b1b9102da468e77127f1e69d6
```

27°C Mostly cloudy 12:04 PM 10/2/2026

My Reports | M | X patient_report_ X patient_report_ X patient_report_ X Hash Calculator: X Password Crac: X Password Crac: X Password Crac: X + -

https://networkwalks.com/hash-calculator/

Text File PDF

Click to choose a PDF, or drop one here
Nothing is uploaded, the PDF is parsed locally

patient_report_1.pdf | 3.6 KB

This PDF is encrypted. A crackable hash has been extracted below (pdf2john / hashcat compatible format).

PDF Hash

```
$pdf$2*3*128*4294967292*1*32*3361663365326235643333353531613238303137316238333
238373763353339*32*ef16c52ab8efce2c18c79e9d28895b5928bf4e5e4e758a4164004e56ffa
a0108*32*c431fab9cc5ef7b59c244b61b745f71ac5ba427b1b9102da468e77127f1e69d6
```

Copy Download

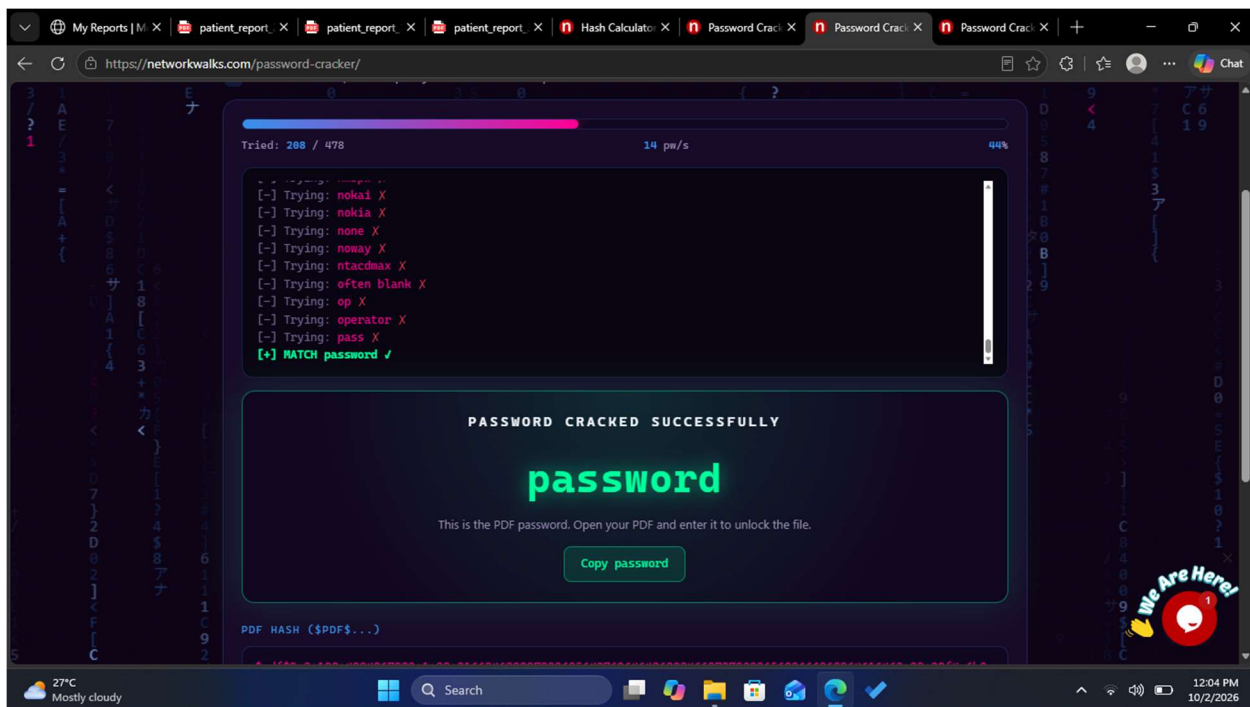
Revision: R3 Version: V2 Key length: 128 bit

Upload a PDF to check if it is password protected and extract a crackable hash.

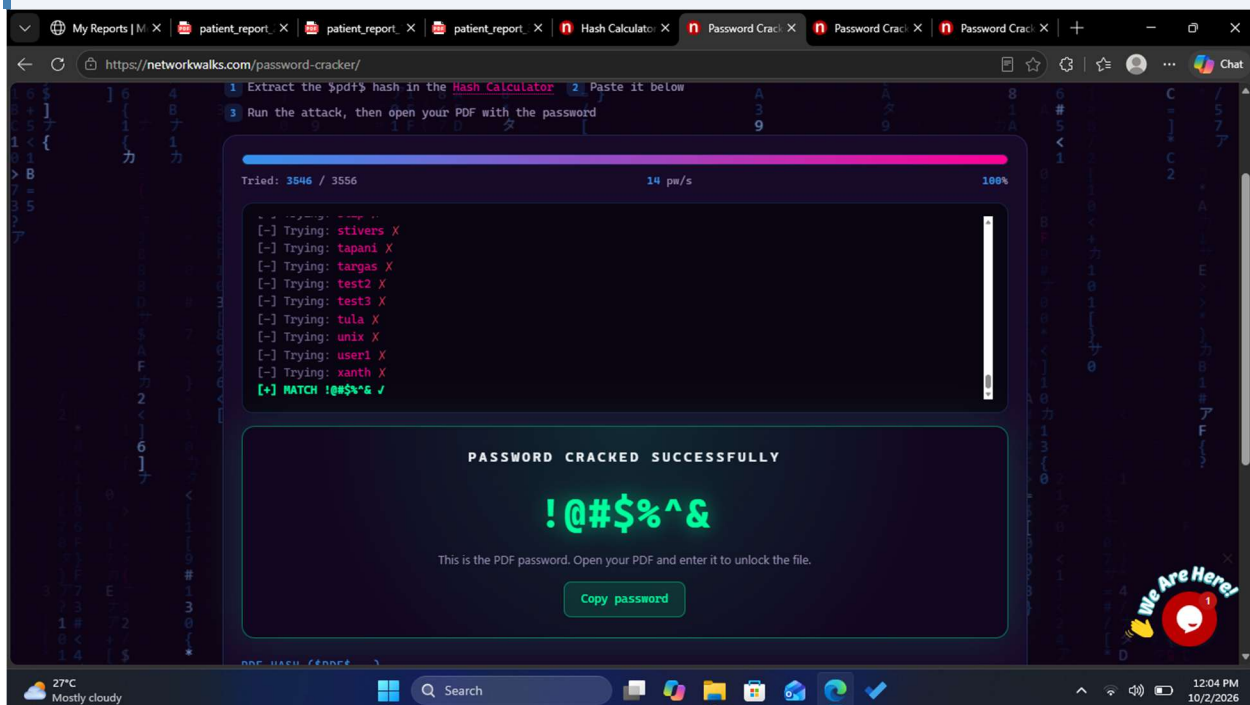
Free tool by Networkwalks Academy | Cybersecurity & Ethical Hacking with AI

whatsapp group to receive study materials & updates:
<https://chat.whatsapp.com/FoLJ5rPXjU16urW4c9MIVV>

27°C Mostly cloudy 12:04 PM 10/2/2026



[Screenshot: Networkwalks Password Cracker cracking Report 3 password using JTR wordlist]



[Decrypted PDF document opened displaying confidential patient medical data]

My Reports | patient_report | patient_report | patient_report | 1 new message | Password Crac | Password Crac | Password Crac |

File D:\Essentials\Downloads\patient_report_2.pdf

1 of 1

Chat

+

Mediroza

General Hospital

14 Cedar Avenue, Johannesburg, South Africa
Tel: +27 11 555 0100
medirozahospital.com

PATHOLOGY LABORATORY REPORT

CONFIDENTIAL

Patient Name: Priya Reddy

Patient ID: MG-P-10244

Date of Birth: 1991-02-28

Gender: Female

Report Date: 2024-11-05

Referring Dr: Dr. Johan van der Merwe

Specimen: Serum

Lab Ref No: LR-2024-1192

TEST	RESULT	UNIT	REFERENCE RANGE	FLAG
Total Cholesterol	6.4	mmol/L	< 5.0	HIGH
LDL Cholesterol	4.1	mmol/L	< 3.0	HIGH
HDL Cholesterol	1.2	mmol/L	> 1.0	Normal
Triglycerides	1.8	mmol/L	< 1.7	HIGH
Fasting Glucose	4.9	mmol/L	3.9 - 5.5	Normal

+

Mediroza

General Hospital

14 Cedar Avenue, Johannesburg, South Africa
Tel: +27 11 555 0100
medirozahospital.com

PATHOLOGY LABORATORY REPORT

CONFIDENTIAL

Patient Name: Sipho Dlamini

Patient ID: MG-P-10231

Date of Birth: 1984-06-12

Gender: Male

Report Date: 2024-11-04

Referring Dr: Dr. Anita Naicker

Specimen: Whole blood (EDTA)

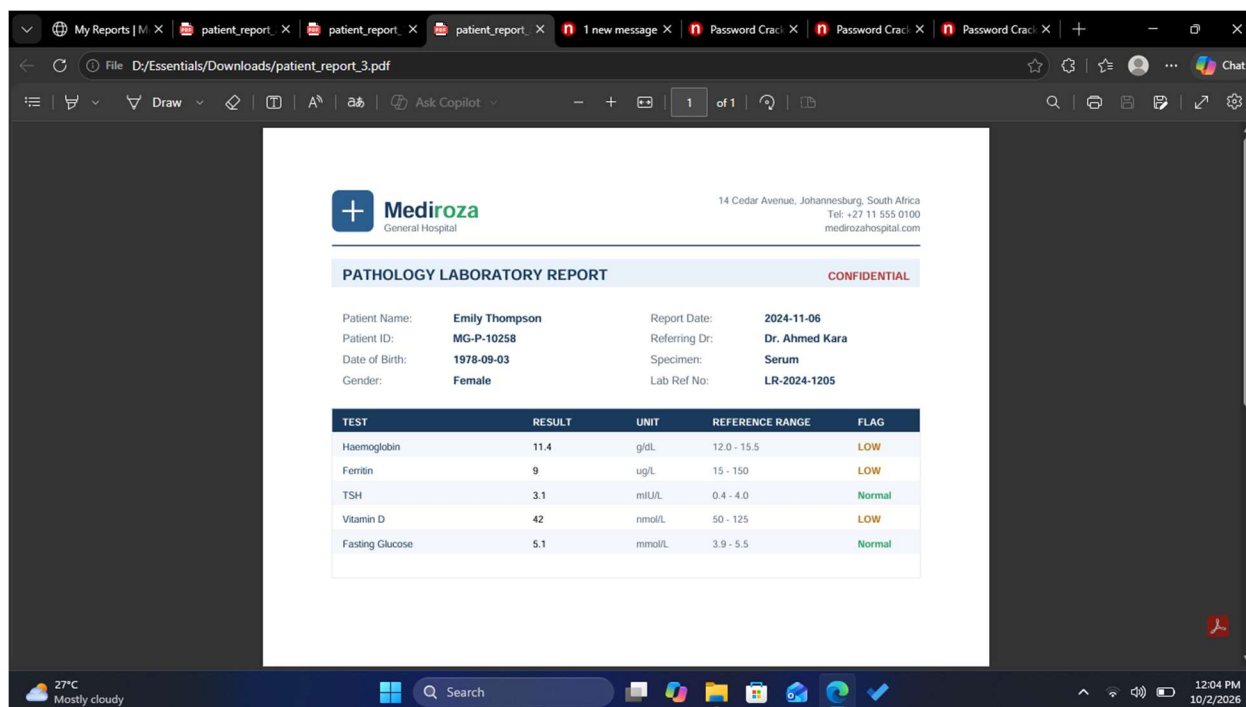
Lab Ref No: LR-2024-1187

TEST	RESULT	UNIT	REFERENCE RANGE	FLAG
Haemoglobin	13.9	g/dL	13.0 - 17.0	Normal
White Cell Count	11.8	x10 ⁹ /L	4.0 - 11.0	HIGH
Platelet Count	245	x10 ⁹ /L	150 - 400	Normal
Fasting Glucose	5.4	mmol/L	3.9 - 5.5	Normal
Creatinine	88	umol/L	64 - 104	Normal

27°C Mostly cloudy

Search

12:04 PM 10/2/2026



3.6 Finding 5 — Sensitive Metadata Left Inside Patient PDF Files

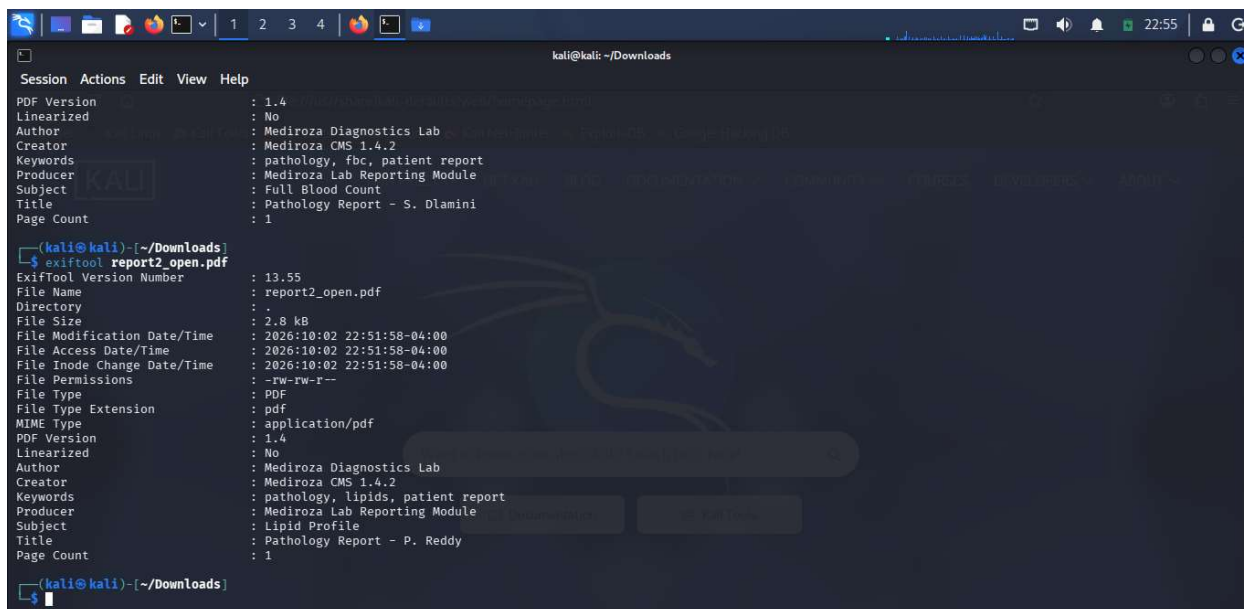
Risk Rating: Medium | **Location:** patient_report_3.pdf

Description: PDF documents store hidden EXIF metadata containing author names, software details, and internal administrative notes. Analyzing decrypted files revealed internal comments left by IT personnel directing users to a database backup location.

Steps Taken:

1. Decrypted Report 3 using qpdf: `qpdf --password='!@#$$%^&' --decrypt patient_report_3.pdf report3_open.pdf`
2. Inspected document metadata attributes using ExifTool: `exiftool report3_open.pdf`
3. Extracted critical metadata fields:
 - Author: J. Malik (Dr. Jameel Malik, IT Systems Administrator)
 - Comment: 'DB backup moved to /old before site migration, do not delete'

[Screenshot: ExifTool command output displaying Author: J. Malik and sensitive Comment field]



3.7 Finding 6 — Forgotten Backup Folder with Directory Listing Enabled

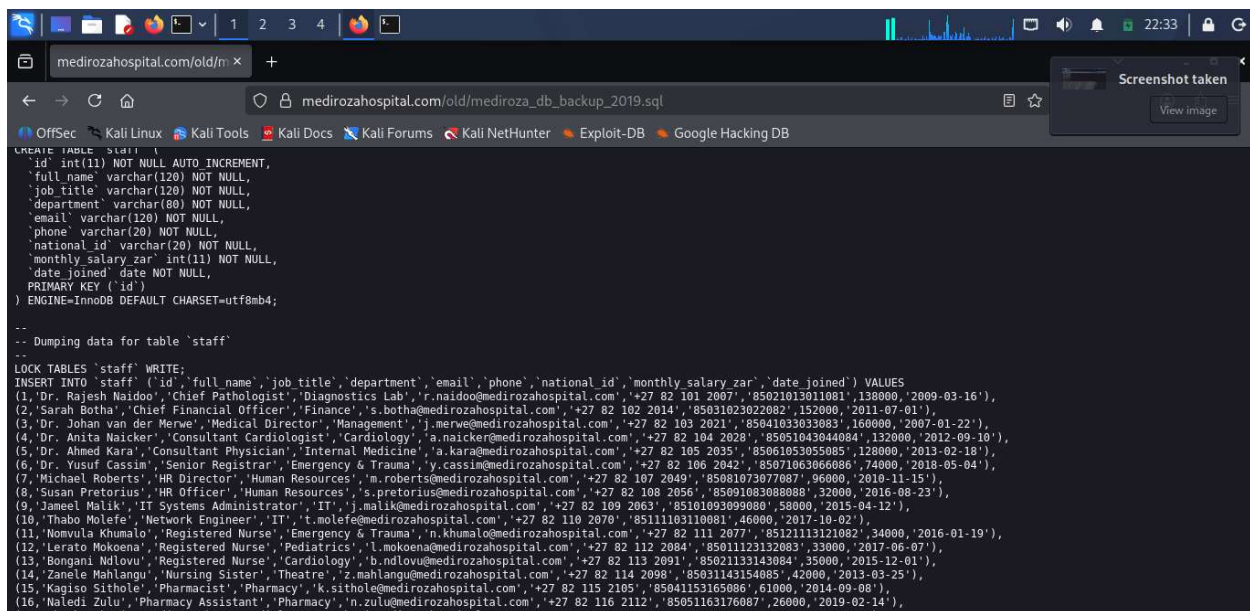
Risk Rating: Critical | **Location:** /old/

Description: Directory listing is a web server misconfiguration that renders directory contents as an indexable file browser. The /old folder—previously identified as a disallowed path in /robots.txt—had directory indexing enabled, exposing administrative backups.

Steps Taken:

1. Navigated to <https://medirozahospital.com/old/> based on robots.txt recon and ExifTool metadata leads.
2. Verified directory indexing was active, displaying mediroza_db_backup_2019.sql.
3. Downloaded the backup file directly using wget: `wget https://medirozahospital.com/old/mediroza_db_backup_2019.sql`

[Screenshot: Browser displaying /old/ directory index with mediroza_db_backup_2019.sql visible]



The screenshot shows a web browser window with the address bar displaying `medirozahospital.com/old/m`. The page content shows a file named `medirozahospital.com/old/mediroza_db_backup_2019.sql`. A terminal window is open, displaying the SQL backup content. The SQL code includes a `CREATE TABLE` statement for a table named `staff`, followed by an `INSERT INTO` statement with 16 rows of data. The data includes staff names, titles, departments, emails, phone numbers, national IDs, monthly salaries in ZAR, and dates joined. A 'Screenshot taken' notification is visible in the top right corner of the browser window.

```
CREATE TABLE `staff` (
  `id` int(11) NOT NULL AUTO INCREMENT,
  `full_name` varchar(120) NOT NULL,
  `job_title` varchar(120) NOT NULL,
  `department` varchar(80) NOT NULL,
  `email` varchar(120) NOT NULL,
  `phone` varchar(20) NOT NULL,
  `national_id` varchar(20) NOT NULL,
  `monthly_salary_zar` int(11) NOT NULL,
  `date_joined` date NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

--
-- Dumping data for table `staff`
--
LOCK TABLES `staff` WRITE;
INSERT INTO `staff` (`id`,`full_name`,`job_title`,`department`,`email`,`phone`,`national_id`,`monthly_salary_zar`,`date_joined`) VALUES
(1,'Dr. Rajesh Naidoo','Chief Pathologist','Diagnostics Lab','r.naidoo@medirozahospital.com','+27 82 101 2007','85021013011081',138000,'2009-03-16'),
(2,'Sarah Botha','Chief Financial Officer','Finance','s.botha@medirozahospital.com','+27 82 102 2014','85031023022082',152000,'2011-07-01'),
(3,'Dr. Johan van der Merwe','Medical Director','Management','j.merwe@medirozahospital.com','+27 82 103 2021','85041033033083',160000,'2007-01-22'),
(4,'Dr. Anita Naicker','Consultant Cardiologist','Cardiology','a.naicker@medirozahospital.com','+27 82 104 2028','85051043044084',132000,'2012-09-10'),
(5,'Dr. Ahmed Kara','Consultant Physician','Internal Medicine','a.kara@medirozahospital.com','+27 82 105 2035','85061053055085',128000,'2013-02-18'),
(6,'Dr. Yusuf Cassim','Senior Registrar','Emergency & Trauma','y.cassim@medirozahospital.com','+27 82 106 2042','85071063066086',74000,'2018-05-04'),
(7,'Michael Roberts','HR Director','Human Resources','m.roberts@medirozahospital.com','+27 82 107 2049','85081073077087',96000,'2019-11-15'),
(8,'Susan Pretorius','HR Officer','Human Resources','s.pretorius@medirozahospital.com','+27 82 108 2056','85091083088088',32000,'2016-08-23'),
(9,'Jameel Malik','IT Systems Administrator','IT','j.malik@medirozahospital.com','+27 82 109 2063','85101093099089',58000,'2015-04-12'),
(10,'Thabo Molefe','Network Engineer','IT','t.molefe@medirozahospital.com','+27 82 110 2070','85111103110081',46000,'2017-10-02'),
(11,'Nomvula Khumalo','Registered Nurse','Emergency & Trauma','n.khumalo@medirozahospital.com','+27 82 111 2077','85121113121082',34000,'2016-01-19'),
(12,'Lerato Mokoena','Registered Nurse','Pediatrics','l.mokoena@medirozahospital.com','+27 82 112 2084','85011123132083',33000,'2017-06-07'),
(13,'Bongani Ndlovu','Registered Nurse','Cardiology','b.ndlovu@medirozahospital.com','+27 82 113 2091','85021133143084',35000,'2015-12-01'),
(14,'Zanele Mahlangu','Nursing Sister','Theatre','z.mahlangu@medirozahospital.com','+27 82 114 2098','85031143154085',42000,'2013-03-25'),
(15,'Kagiso Sithole','Pharmacist','Pharmacy','k.sithole@medirozahospital.com','+27 82 115 2105','85041153165086',61000,'2014-09-08'),
(16,'Naledi Zulu','Pharmacy Assistant','Pharmacy','n.zulu@medirozahospital.com','+27 82 116 2112','85051163176087',26000,'2019-02-14'),
(17,'Thobeka Nkomo','Pharmacy Assistant','Pharmacy','t.nkomo@medirozahospital.com','+27 82 117 2119','85061173187088',41000,'2016-07-30');
```

3.8 Finding 7 — Confidential Staff Salaries, Bank Details and Shareholder Data in Plain Text

Risk Rating: Critical | **Location:** /old/mediroza_db_backup_2019.sql

Description: The retrieved SQL backup contained unencrypted records detailing private staff credentials, monthly compensation, bank details, and shareholder equity breakdowns. Exposing this information directly violates PCI-DSS / data privacy mandates.

Steps Taken:

1. Opened the SQL file in a text editor to analyze structure.
2. Extracted raw INSERT statements containing 30 hospital employee records and 10 shareholder profiles.
3. Processed raw SQL data using ChatGPT to render structured staff salary tables (including name, title, department, salary in ZAR) and shareholder equity tables (including name, share percentage, class).
4. Cross-referenced staff records to confirm 'J. Malik' corresponds to Jameel Malik, IT Systems Administrator, completing the full attack chain.

4. Complete Attack Chain Summary

The following step-by-step breakdown details the end-to-end exploit chain executed from initial web reconnaissance to complete database exfiltration:

- Step 1: Analyzed /robots.txt during initial reconnaissance, identifying disallowed endpoints including /patient, /staff, and /old.

- Step 2: Discovered username enumeration on patient/login.php, confirming the existence of the 'admin' account (Finding 1).
- Step 3: Triggered dynamic SQL syntax errors by appending a single quote (') to the login parameter, confirming SQL injection vulnerability (Finding 2).
- Step 4: Applied the comment payload admin' -- to bypass authentication entirely and secure administrative portal access (Finding 2).
- Step 5: Discovered and downloaded three password-protected patient lab report PDF files from patient/reports/ (Finding 3).
- Step 6: Extracted hashes via Networkwoks Hash Calculator and cracked PDF passwords using Networkwalks Password Cracker ('123456', 'password', '!@#\$%^&') (Finding 4).
- Step 7: Decrypted Report 3 and ran ExifTool, revealing a hidden comment by J. Malik pointing to the database backup location in /old (Finding 5).
- Step 8: Accessed https://medirozahospital.com/old/, leveraged directory listing to locate mediroza_db_backup_2019.sql, and downloaded it via wget (Finding 6).
- Step 9: Extracted and structured plain-text employee salaries, bank account details, and shareholder equity data for the entire hospital organization (Finding 7).

5. Recommendations and Remediation

5.1 Fix Username Enumeration

Modify authentication endpoints to return identical generic error messages (e.g., 'Invalid username or password') regardless of whether the user account or password was incorrect.

5.2 Fix SQL Injection (High Priority)

Replace dynamic SQL queries with parameterized queries or prepared statements across all web forms. This isolates executable code from user parameters:

// PHP PDO Example:

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE username = ? AND password = ?');
$stmt->execute([$username, $password]);
```

5.3 Secure PDF Access and Enforce Password Policies

Move confidential patient reports outside the public web root. Serve files strictly through server-side authorization scripts. Enforce minimum 12-character document password policies combining uppercase, lowercase, numbers, and symbols.

5.4 Strip PDF Metadata

Automate metadata removal prior to releasing files externally to prevent information leaks:

```
exiftool -all= patient_report_3.pdf
```

5.5 Disable Directory Listing and Remove Unsecured Backups

Disable directory indexing server-wide by adding 'Options -Indexes' to Apache/.htaccess configuration. Immediately delete all backup files from /old and store backups exclusively in secure, non-web-accessible locations.

6. Conclusion

This assessment demonstrated a complete attack chain originating from a basic authentication bypass and culminating in the full exposure of internal medical, financial, and corporate records. The identified vulnerabilities are well-understood and can be remediated using standard security controls. Remediation of Critical and High findings must be executed immediately before live patient records are managed on this system.

Submitted by: Adzido Godsway

Role: Junior Penetration Tester, NetworkWalks

Target Assessed: <https://medirozahospital.com>